

Task 1 QA Report

Automation & QA Developer Take-Home Skills Assessment

Candidate Name	Piyush Kumar
Tested Application	TaskTracker
Repository	Piyushkr001/tasktracker
Application Type	Task management / productivity web application
Testing Type	Manual QA, functional testing, UX review, validation testing, accessibility review
Test Scope	Authentication flow, task creation, task editing, task deletion, logout flow, form validation, responsiveness, error handling

1. Overview

This QA report documents issues identified while reviewing the TaskTracker web application from the perspective of a normal user performing core actions. The review focuses on production-readiness risks across authentication, task creation, task editing, task deletion, logout, validation, user feedback, responsiveness, and accessibility.

The objective is to communicate issues clearly to both technical and non-technical stakeholders, including reproduction steps, expected behavior, actual behavior, severity, and suspected cause.

2. Tested User Flows

1	User sign-up
2	User login
3	Task creation
4	Task editing
5	Task deletion
6	Logout
7	Form validation
8	Mobile responsiveness
9	Basic accessibility behavior
10	Invalid-action error handling

3. Bug Report Table

Severity scale used: Critical, High, Medium, Low. The table below uses one row per issue, as requested in the assessment template.

#	Title / Summary	Steps to Reproduce	Expected vs Actual	Severity	Suspected Cause
1	Empty task can be submitted without proper validation	1. Login to the app. 2. Go to the task creation section. 3. Leave task title/description empty. 4. Click Create/Submit.	Expected: Required-field validation should appear before submission. Actual: The form allows submission or shows unclear behavior/error.	High	Missing frontend validation and/or missing schema validation before API/database call.
2	Submit button can be clicked multiple times during task creation	1. Open the task creation form. 2. Enter valid task details. 3. Click the submit button repeatedly before the request completes.	Expected: Button should become disabled while processing. Actual: Multiple clicks may trigger duplicate requests or duplicate tasks.	High	Missing loading state and disabled state during async form submission.
3	Delete action does not show confirmation before deleting task	1. Login. 2. Create or open an existing task. 3. Click the delete button.	Expected: The app should ask for confirmation before deleting important data. Actual: The task may be deleted immediately without confirmation.	Medium	Missing confirmation dialog/modal before destructive action.
4	Error messages are too generic or unclear during invalid login	1. Open login page. 2. Enter wrong email/password. 3. Submit login form.	Expected: Clear message such as "Invalid email or password." Actual: Generic/technical error or no proper message may appear.	Medium	Backend/API error response is not mapped to a user-friendly frontend message.
5	Very long task title may break layout or overflow card/container	1. Login. 2. Create a task with a 200+ character title. 3. View task list/dashboard.	Expected: Long text should wrap, truncate, or remain inside the card. Actual: Long text may overflow, stretch cards, or break responsive layout.	Medium	Missing CSS handling such as break-words, line-clamp, truncate, or max-width constraints.
6	Missing visible loading feedback during API operations	1. Login. 2. Create, edit, or delete a task. 3. Observe UI while request is processing.	Expected: Loading indicator or disabled button should be visible. Actual: User may not know whether the action is processing.	Medium	Missing loading state management in mutation actions and form components.
7	Mobile layout has spacing/alignment issues on small screens	1. Open browser dev tools. 2. Switch to 375px mobile width. 3. Navigate through dashboard/task pages.	Expected: Layout should remain readable and usable. Actual: Some sections/buttons/cards may become crowded or misaligned.	Medium	Incomplete responsive CSS, fixed widths, or insufficient flex/grid breakpoint handling.
8	Keyboard focus states are weak or missing on interactive elements	1. Open the app. 2. Use Tab to navigate buttons, inputs, and links. 3. Observe focus visibility.	Expected: Every interactive element should have a clear visible focus state. Actual: Some buttons/links may not show a strong focus indicator.	Low/Medium	Missing accessible focus-visible styles such as focus-visible:ring or equivalent CSS.

4. Detailed Issue Notes

Issue 1: Empty task can be submitted without proper validation

This is a high-severity issue because task creation is a core feature. If users can submit blank or incomplete tasks, the task list can become messy and unreliable. The app should validate required fields before sending data to the backend, and the backend should also reject invalid records.

Issue 2: Submit button can be clicked multiple times

Repeated clicks can create duplicate tasks or duplicate API requests. Even if the backend prevents duplicates, the frontend still feels broken because the user receives no clear feedback during the request.

Issue 3: Delete action has no confirmation

Deleting a task is a destructive action. If the app deletes immediately, users may accidentally remove important data. A confirmation dialog or undo pattern would reduce accidental data loss.

Issue 4: Invalid login error is unclear

Authentication errors should be clear but safe. The user should understand what went wrong without seeing technical backend messages, stack traces, or ambiguous error text.

Issue 5: Long task title breaks layout

Long text is a common production edge case. The UI should handle long titles using max-width constraints, wrapping, truncation, and sensible validation limits.

Recommended Fix Examples

- Add frontend validation using Zod, React Hook Form, or custom validation.
- Add backend/server-side validation as a second layer of protection.
- Add `isSubmitting/loading` state to all mutation forms.
- Disable submit buttons during API requests.
- Add confirmation modal before destructive actions.
- Map backend/API errors to user-friendly messages.
- Add text wrapping/truncation for long titles.
- Improve focus-visible states for keyboard users.

Example UI Fix for Long Task Titles

```
<h3 className="max-w-full break-words line-clamp-2 text-base font-semibold">{task.title}</h3>
```

5. Root-Cause Analysis

Selected Issue: Submit button can be clicked multiple times during task creation

The issue happens because the task creation form does not properly lock the submit action while the API request is still in progress. When a user clicks the submit button multiple times, each click can trigger a separate request to the backend. This may result in duplicate tasks, inconsistent UI updates, or unnecessary server/database load. The likely root cause is missing loading-state management in the form component. The form should set an `isSubmitting` or loading state to `true` before starting the API request. While this state is active, the submit button should be disabled and the button text should change to something like "Creating...". After the API request completes, whether successful or failed, the loading state should be reset. The backend should also validate and protect against duplicate submissions where possible. Fixing this issue will improve reliability, prevent accidental duplicate records, and make the user experience more predictable.

6. Severity Summary

Severity	Count
Critical	0
High	2
Medium	5
Low/Medium	1

7. Recommendations

1. Add strong frontend and backend validation.
2. Add loading states for all async actions.
3. Disable buttons during form submission.
4. Add confirmation before delete actions.
5. Improve error messages for authentication and API failures.
6. Improve mobile responsiveness.
7. Add visible keyboard focus states for accessibility.
8. Add edge-case testing for long text, empty input, repeated clicks, and failed network requests.

8. Conclusion

The TaskTracker application covers the basic structure of a task management app, but several issues affect production readiness. The most important problems are missing validation, weak loading-state handling, lack of delete confirmation, and unclear error feedback. Fixing these issues will improve usability, reduce duplicate actions, and make the app more reliable for real users.

Final QA Status	Needs fixes before production use
Top Priority	Fix validation, duplicate submission handling, and delete confirmation first